《钱包请等等 / Buy or Bye》数据库设计文档

项目：理性消费决策 App（仿小程序重做为 Android 原生应用）

技术栈：Spring Boot（Java）· Android Studio · MySQL 8.4 / InnoDB / utf8mb4

表规模：30 张（P0 必做 19 张 · P1 建议 8 张 · P2 可选 3 张）

一、设计目标与约束

1.1 这个数据库要解决什么问题

产品只有一件事：**用户想买一个东西但拿不定主意，把它发出去让别人投票，最后得到"买 / 不买 / 选哪个"的结论。**

听起来简单，但它同时牵涉五个互相牵连的问题：

问题	落到数据库上
谁在问、谁在答	用户身份 + 群组权限
问的是"买不买"还是"选哪个"	两套投票模型共用一张表
答完怎么算结果	结算状态机 + 票权快照
结论要能追溯、能上榜	信誉流水 + 汇率快照
手机端弱网怎么保证不重复	幂等键

数据库设计的全部复杂度，都来自这五个问题的交叉。

1.2 硬约束

约束	影响
MySQL 8.4 / InnoDB	支持 CHECK 约束、窗口函数，可以用 `DECIMAL` 精确金额
前后端分离	数据库只服务后端，Android 不直连，所有访问经 REST API
全 Java	字段类型映射到 `Long` / `BigDecimal` / `LocalDateTime` / `enum`
课程项目（5 人 · 一学期）	30 张表分 P0/P1/P2 三档，允许裁剪

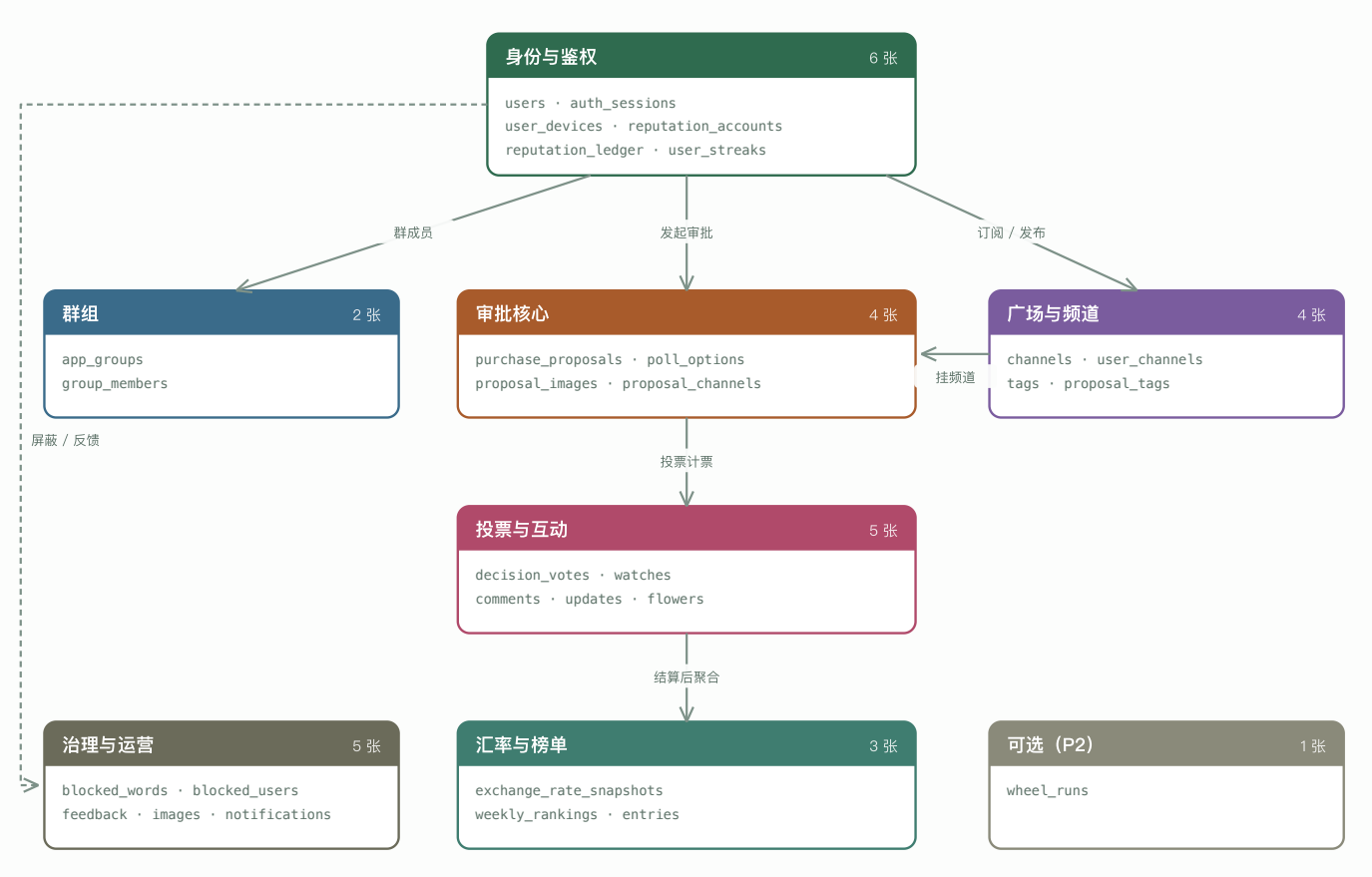
1.3 六条设计原则

1. **事实与汇总分离**：投票是事实（`decision_votes`），表态人数是汇总（`voter_count`）。事实表保证正确，汇总字段保证列表页不卡。
2. **快照优先于实时计算**：投票时冻结票权，结算时冻结汇率。规则以后怎么改，历史结论都不会变。
3. **状态用字符串不用数字**：`SETTLED` 比 `3` 可读，也避免"3 到底代表什么"的团队沟通成本。
4. **金额绝不使用浮点**：全库 `DECIMAL`，Java 侧 `BigDecimal`。
5. **数据库只管单行约束，跨行规则交给服务层**：比如"一条审批最多 3 个标签"，SQL 表达不了区间，必须 Spring 事务里校验。

6. **能算出来的不存，除非性能不允许**：等级由分数推导（不存），但表态人数要存（列表页每行都 `COUNT` 会拖垮分页）。

二、整体架构：按业务域分治

30 张表不是平铺的，而是围绕 **8 个业务域**组织。分域的依据是"**变更频率**"——身份、审批核心几乎不改；榜单、转盘随时可能调整。把易变的隔离出去，改一个域不会牵动全身。



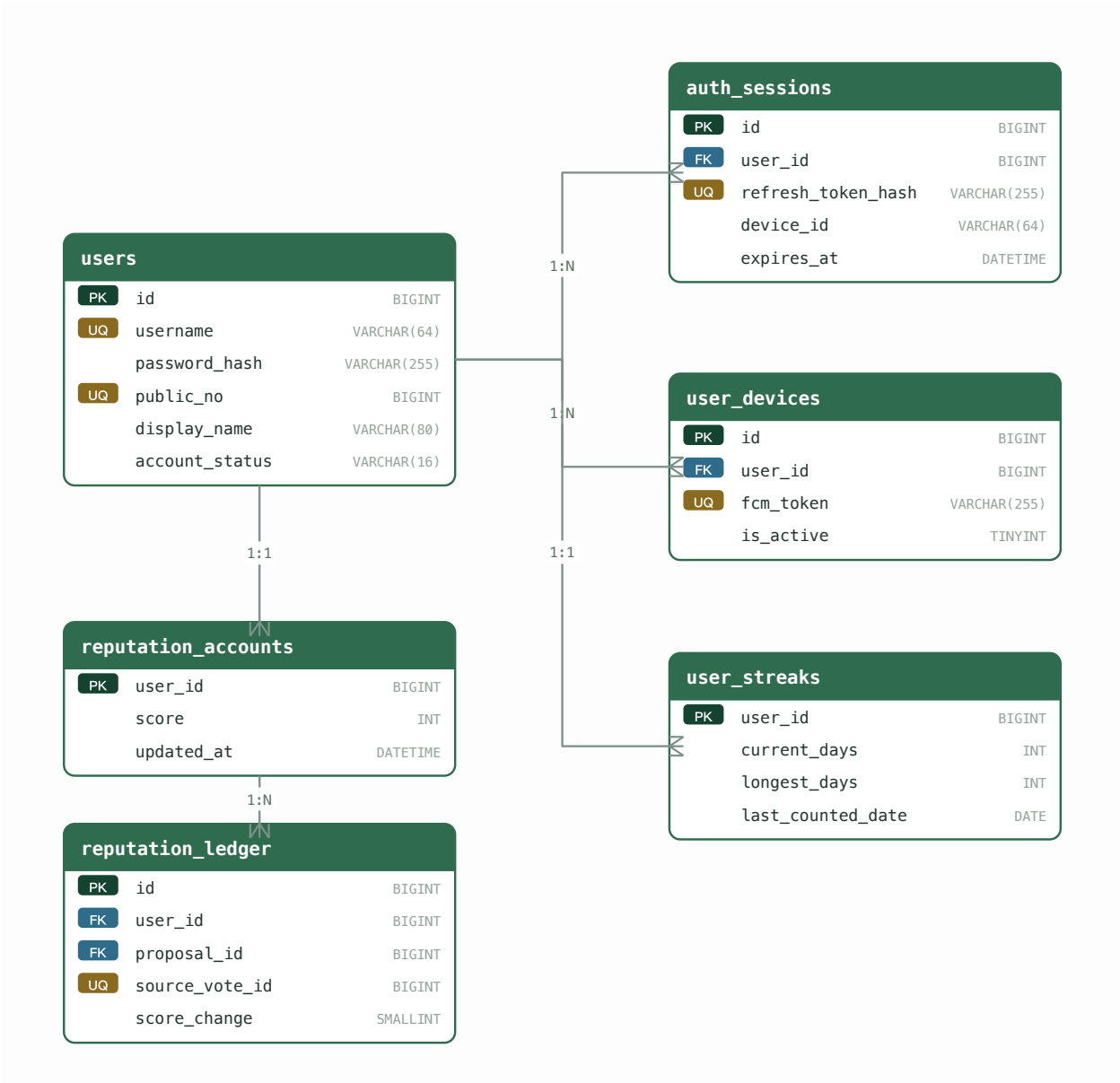
域	表数	这个域回答什么问题
身份与鉴权	6	你是谁、怎么登录、你的信誉和坚持天数是多少
群组	2	你能在哪个圈子里发起和参与审批
审批核心	4	一条审批长什么样、有几个选项、发到哪
广场与频道	4	公开内容怎么分类、用户订阅了哪些
投票与互动	5	别人怎么表态、怎么围观、怎么回应、怎么送花
汇率与榜单	3	跨币种金额怎么比、周榜怎么冻结
治理与运营	5	屏蔽、拉黑、反馈、通知
可选（P2）	1	转盘历史（默认不建）

三、领域详解

3.1 身份与鉴权域（6 张）

这个域解决什么：让一个真实的人变成系统里的一个 `user_id`，并保证他的登录态安全、可撤销，同时承载信誉和"坚持天数"这两个激励指标。

为什么单独成域：其他所有域都通过 `user_id` 指回来。这个域一旦改动，全库受影响，所以它的表要最稳定、最少变。



users —— 账号与公开资料

- **用途**：存注册账号、登录凭证（BCrypt 哈希）和对外展示资料（昵称、头像、公开 ID）。
- **为什么需要这张表**：整个系统唯一的身份锚点。审批、投票、群成员、信誉、屏蔽、反馈——所有行为都必须能归属到某个人，全部通过 `user_id` 指回这里。没有它，其他 29 张表都失去主语。

- **关键关系**：1:N → `auth_sessions` / `user_devices` / `reputation_ledger` / `blocked_words` / `blocked_users` / `feedback` / `notifications` / `flowers`；1:1 → `reputation_accounts` / `user_streaks`
- **核心约束**：`UNIQUE(username)`、`UNIQUE(public_no)`

auth_sessions —— 登录会话

- **用途**：记录每一次设备登录，管理 Refresh Token 的签发、续期与吊销。
- **为什么需要这张表**：这是 Android 相比小程序多出来的一层。小程序由微信托管登录态，安卓必须自己管。没有它，只能发一个**永不过期的 token**——用户改密码、手机丢失后无法强制下线，这是明确的安全漏洞。
- **关键关系**：N:1 → `users`（一个用户可多设备）
- **核心约束**：`UNIQUE(refresh_token_hash)`；`revoked_at` 是"踢下线"的唯一依据

user_devices —— 推送设备

- **用途**：存 FCM 注册 token，用于向指定设备推送"你的审批有结果了""有人回应你"。
- **为什么需要这张表**：安卓的 FCM token 会因为**重装、清数据、换机**而失效。如果只存在用户表的一个字段里，换机后旧 token 会一直失败重试。必须支持一人多设备 + token 轮换 + 失效标记。
- **关键关系**：N:1 → `users`
- **核心约束**：`UNIQUE(fcm_token)`；FCM 返回 `UNREGISTERED` 时置 `is_active = 0`

reputation_accounts —— 信誉分汇总

- **用途**：存每个用户当前的信用分，用于计算票权（1/2/3）和个人页展示。
- **为什么需要这张表**：信誉分是**每次投票都要读**的数据（要算票权）。如果每次都去流水表求和，一次结算要扫上千行。这里存快照，流水表负责审计，各司其职。
- **关键关系**：1:1 → `users`（`user_id` 既是主键也是外键，天然保证一人一行）
- **核心约束**：等级/徽章由分数区间推导，**不落库**，避免两处真相

reputation_ledger —— 信誉分流水

- **用途**：记录每一次信誉加减的明细：谁、因为哪一票、加了几分、用的哪版规则。
- **为什么需要这张表**：只有汇总没有流水，用户问"我为什么少了 5 分"你答不出来。更关键的是**规则会改**——今天"+1 分"，明天可能改成"加权 +2"，没有 `rule_version` 就无法解释历史。这张表还承担幂等职责：`source_vote_id` 唯一，保证同一票不会重复加分。
- **关键关系**：N:1 → `users` / `purchase_proposals`；1:1 → `decision_votes`（`source_vote_id`）
- **核心约束**：`UNIQUE(source_vote_id)` —— 定时任务重复执行也不会重复发分

user_streaks —— 理性消费坚持天数

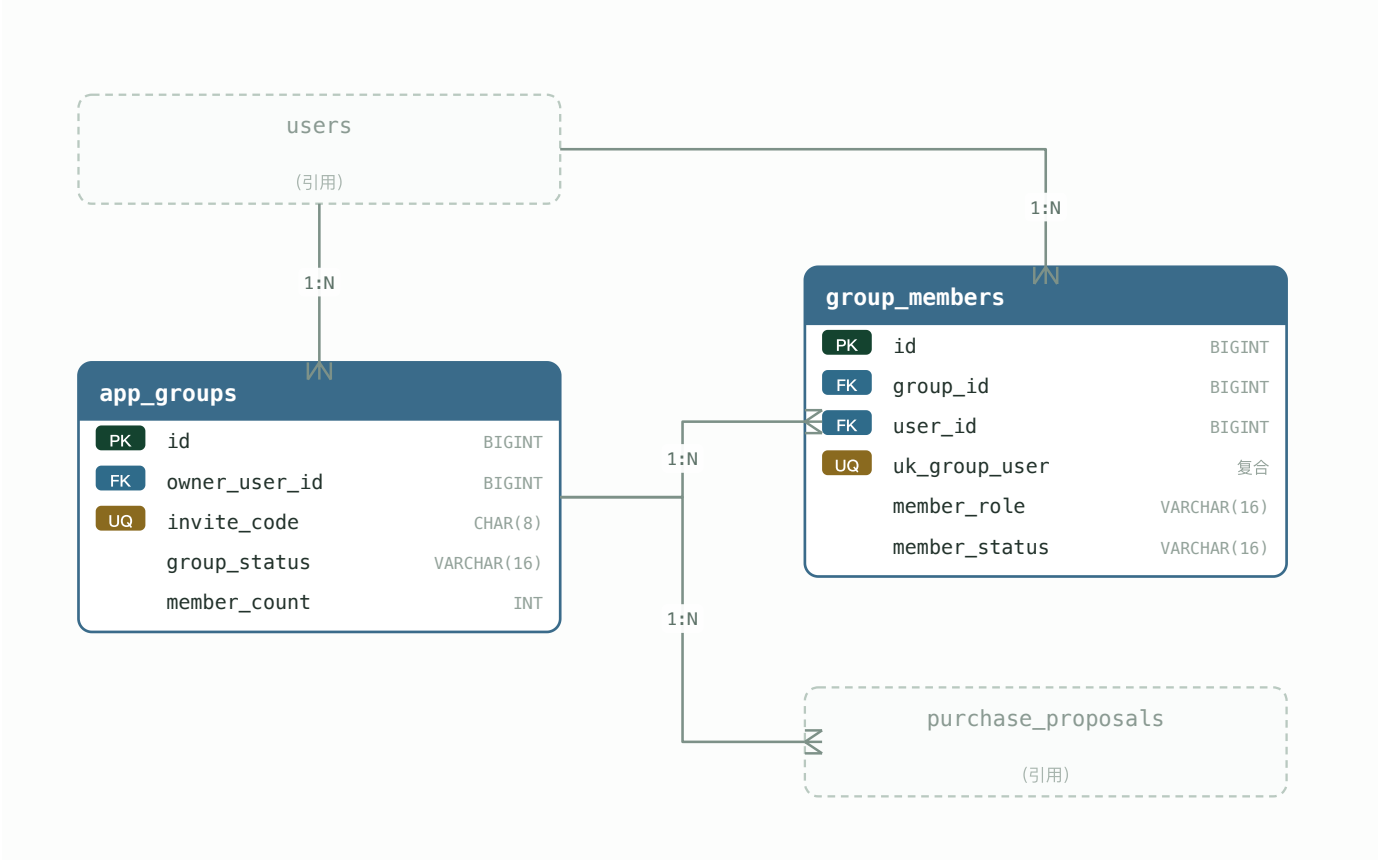
- **用途**：存"已坚持理性消费 N 天"，用于个人页的激励展示。
- **为什么需要这张表**：这个数字**无法用单条 SQL 实时算出**——它需要判断"连续自然日"是否中断，属于窗口函数才能做的序列计算，而且每次进个人页都算一遍成本很高。用一张小表每天定时更新，读写都便宜。
- **关键关系**：1:1 → `users`

- **核心约束**：`last_counted_date` 防止同一天重复计数

3.2 群组域（2 张）

这个域解决什么：把用户组织成小圈子，让审批可以只在熟人之间进行（"别让同事看到我想买第五双球鞋"）。

设计要点：群本身和"谁在群里"必须分开。群是长期存在的实体，成员关系会反复进出——如果合成一张表，成员一退出就得重建群信息。



app_groups —— 审批群

- **用途**：存群的基本信息：群名、群主、8 位群号、状态、成员数。
- **为什么需要这张表**：群是审批的**发布容器**。一条审批要么发到群（`publication_scope = GROUP`），要么发到广场，没有群就没有"只给熟人看"这个选项。另外群号是拉新的入口——"输入群号加入"必须能反查到群。
- **关键关系**：N:1 → `users`（群主）；1:N → `group_members` / `purchase_proposals`
- **核心约束**：`UNIQUE(invite_code)` —— 必须是**可反查的明文短码**，不能用哈希

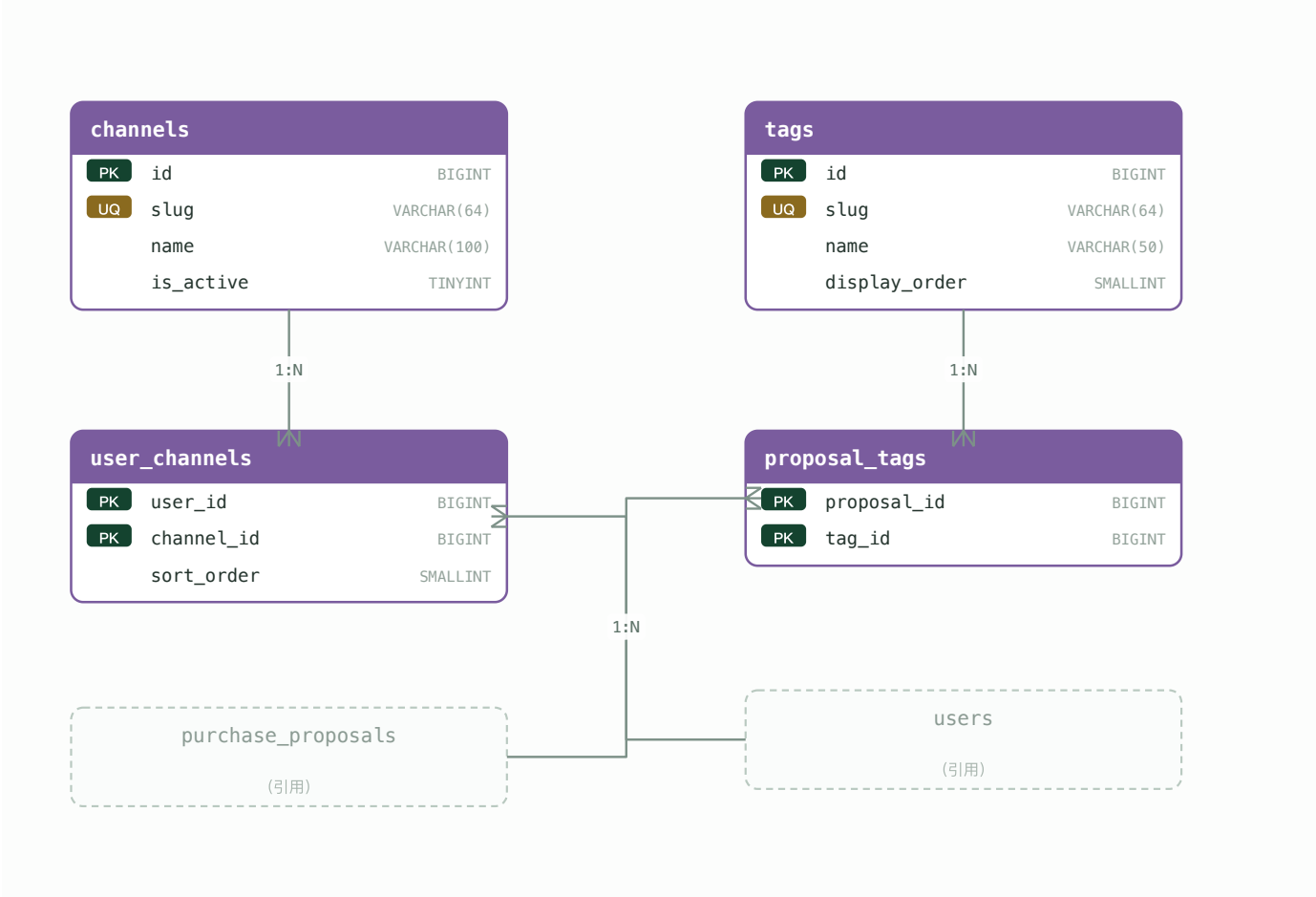
group_members —— 群成员关系

- **用途**：一条记录 = 一个用户在一个群里的身份、角色、加入/退出时间。
- **为什么需要这张表**：它是**群内权限的唯一来源**。判断"我能不能在这个群发起审批""这条审批我有没有资格投票"，全靠这张表。同时它保留了退出记录（`member_status = LEFT` + `left_at`），这样历史投票不会因为退群而失去上下文。
- **关键关系**：N:1 → `app_groups` / `users`
- **核心约束**：`UNIQUE(group_id, user_id)` —— 重进群是**更新原行**而非插新行，投票历史留在 `decision_votes` 不受影响

3.3 广场与频道域（4 张）

这个域解决什么：让审批可以被陌生人看到。广场是公开内容池，频道是内容池的分区，标签是内容的属性。

为什么频道和标签要分成两套：界面上同时存在"我的频道（随便看看 / 奶茶）"和"内容标签（综合 / 食物 / 数码...）"。前者是**运营侧栏目**，用户可订阅、可排序；后者是**内容属性**，发布时必选 1–3 个。两者的生命周期、维护者、查询方式都不同，合表会导致"订阅一个标签"这种语义混乱。



channels —— 广场频道

- **用途**：存运营配置的频道：随便看看、奶茶、数码好物等。
- **为什么需要这张表**：广场不能是一个大杂烩。频道让用户能"只看我关心的"，也让运营能推专题。`slug` 用稳定标识而不是自增 id，是为了让前端路由和分享链接不因换库而断掉。
- **关键关系**：1:N → `user_channels` / `purchase_proposal_channels`
- **核心约束**：`UNIQUE(slug)`

user_channels —— 用户订阅频道

- **用途**：记录用户订阅了哪些频道，以及它们的展示顺序。
- **为什么需要这张表**：对应界面上的"我的频道"和"管理频道"。`sort_order` 是为拖拽排序准备的——如果没有它，用户调整完顺序下次进来又乱了。
- **关键关系**：N:1 → `users` / `channels`（复合主键，天然防重复订阅）

- **核心约束**：`PRIMARY KEY(user_id, channel_id)`

tags —— 内容标签

- **用途**：存 16 个预设标签：综合、生活、食物、穿搭、美妆、数码、追星、二次元、游戏、艺术、旅行、运动、宠物、学习、职场、情感。
- **为什么需要这张表**：发布审批时标签是**必选项**（1–3 个），广场的分类 tab 也直接用它筛选。如果把标签写成审批表里的逗号分隔字符串，就没法做"按标签查全部内容"，也没法统计"哪个标签最活跃"。
- **关键关系**：N:M → `purchase_proposals`（通过 `proposal_tags`）
- **核心约束**：`UNIQUE(slug)`；`display_order` 保证标签栏顺序固定

proposal_tags —— 审批与标签的关联

- **用途**：多对多中间表，记录每条审批打了哪几个标签。
- **为什么需要这张表**：一条审批可以有多个标签，一个标签下有多条审批，这是典型的 N:M，必须用中间表。复合主键同时起到"同一标签不能重复打"的作用。
- **关键关系**：N:1 → `purchase_proposals` / `tags`
- **核心约束**：`PRIMARY KEY(proposal_id, tag_id)`；"1–3 个"的数量限制**只能由 Spring 服务层在事务内校验**

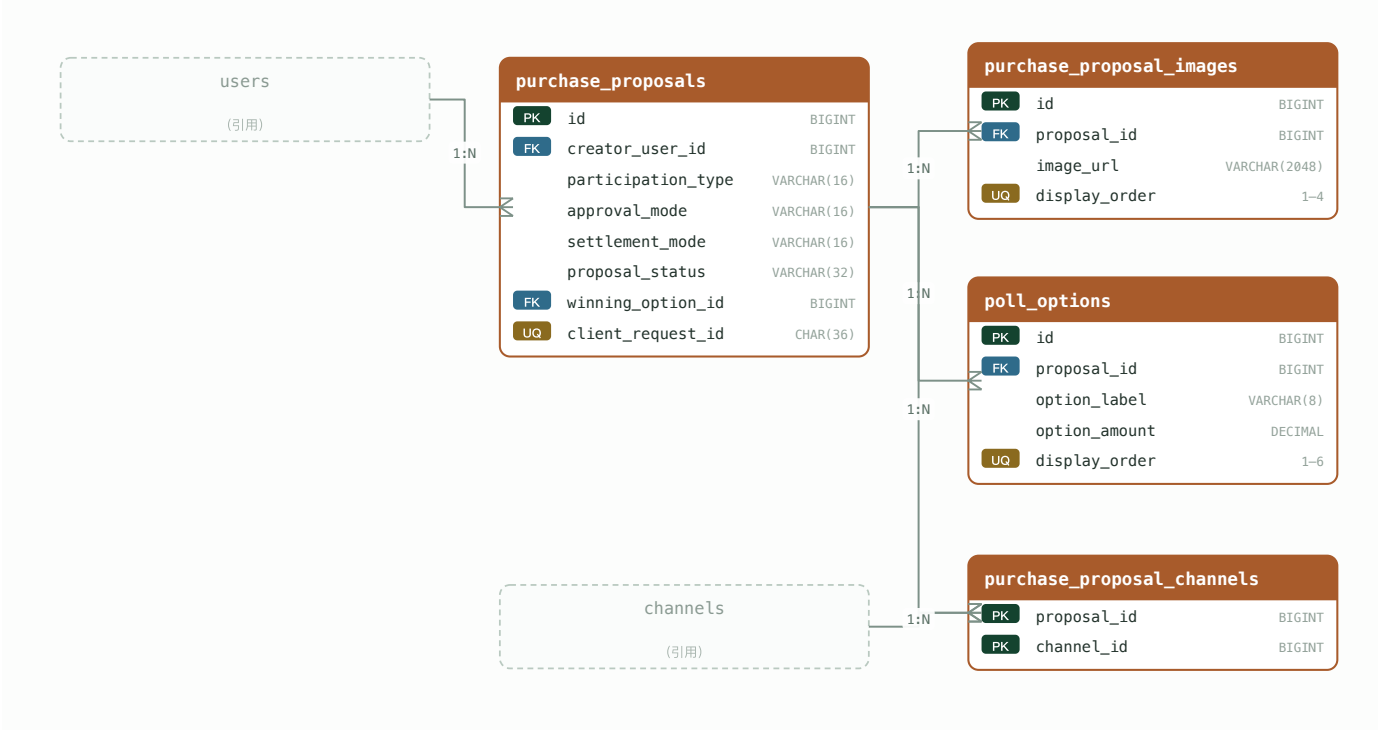
3.4 审批核心域（4 张）

这个域解决什么：一条审批单的完整生命周期——草稿、发布、投票、结算、撤回。

这是全库最核心也最复杂的域，因为它要同时承载两种完全不同的业务形态：

形态	界面	结果怎么表达
二元裁决	"通过 / 驳回"	`decision_result` = 1 或 0
多选一	"选项 A / B / C"	`winning_option_id` 指向某个选项

原设计只考虑了第一种，把结果焊死在 `TINYINT` 上，所以必须补 `participation_type` 和 `winning_option_id`。



purchase_proposals —— 审批单（核心表）

- **用途**：一条记录 = 一次"要不要买 / 选哪个"的决策请求，包含标题、金额、理由、发布范围、状态、时间轴。
- **为什么需要这张表**：它是整个产品的**主实体**。广场信息流、群内审批列表、个人页统计、周榜、汇率快照——全部以它为中心展开。它同时承担草稿存储（`status = DRAFT`），因为草稿和正式审批的字段几乎一致，分两张表会带来大量重复和转换代码。
- **关键关系**：N:1 → `users`（发起人）/ `app_groups`（目标群）；1:N → `poll_options` / `purchase_proposal_images` / `purchase_proposal_channels` / `decision_votes` / `proposal_watches` / `proposal_comments` / `proposal_updates` / `flowers`；1:1 → `exchange_rate_snapshots`
- **核心约束**：发群必须填 `group_id`、发广场必须为空；`SETTLED` 时必须有结果；`client_request_id` 保证弱网重复提交不产生两条

poll_options —— 选项投票的候选项

- **用途**：存"选项投票"模式下的 A/B/C 候选项，每个选项可带独立价格。

- **为什么需要这张表**：没有它，界面上三个页面（填选项、加选项、发布投票）全部无法实现。主表的 `amount` 是单值，存不下"A ¥120 / B ¥89 / C ¥150"。而且这张表还是**转盘的扇区数据源**——转盘不需要另建表，直接读它。
- **关键关系**：N:1 → `purchase_proposals`；1:N → `decision_votes.option_id`
- **核心约束**：`UNIQUE(proposal_id, display_order)`；`CHECK(display_order BETWEEN 1 AND 6)`

purchase_proposal_images —— 审批配图

- **用途**：存审批的配图 URL 和展示顺序，最多 4 张。
- **为什么需要这张表**：图片是一对多关系，塞进主表只能用逗号分隔字符串，没法排序、没法限制数量、也没法单独删除某一张。文件本身放对象存储，这里只存 URL，避免 MySQL 被大字段拖慢备份。
- **关键关系**：N:1 → `purchase_proposals`
- **核心约束**：`UNIQUE(proposal_id, display_order)`，`display_order` 限 1–4

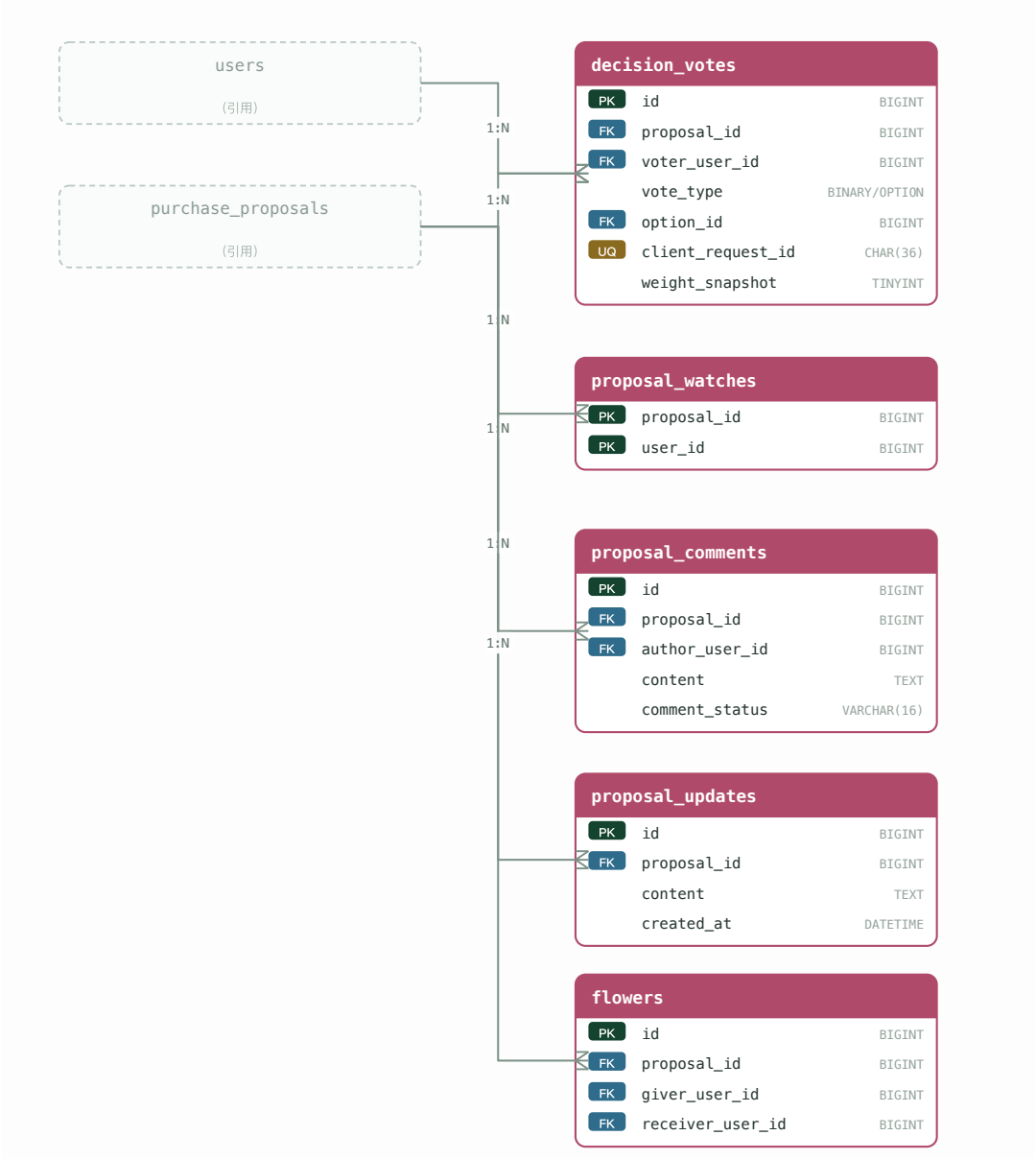
purchase_proposal_channels —— 审批与频道的关联

- **用途**：记录一条广场审批发布到了哪些频道。
- **为什么需要这张表**：一条审批可以同时进"数码"和"随便看看"两个频道。关键设计是：**同一审批出现在多个频道时，投票和结果仍是同一份**，不会因为多渠道而分裂成两个票池。如果直接在审批表存一个 `channel_id`，就做不到多频道。
- **关键关系**：N:1 → `purchase_proposals` / `channels`
- **核心约束**：`PRIMARY KEY(proposal_id, channel_id)`；服务层保证广场审批至少一个频道、群内审批零个频道

3.5 投票与互动域（5 张）

这个域解决什么：别人怎么对一条审批做出反应——表态、围观、回应、送花。

为什么投票和互动要分开：投票**参与计票**，围观和回应**不参与**。如果把围观也塞进 `decision_votes`，结算时统计票数就会把"只是看看"的人算进去，结果直接算错。语义不同的行为必须分表。



decision_votes —— 投票事件

- **用途**：记录每一次表态：谁、对哪条审批、投了什么、当时票权多少。
- **为什么需要这张表**：这是**计票的唯一依据**。设计成**追加写入**（改票是新增一行，只取每人最新一行计票）而不是原地更新，好处是能审计"这个人改过票"、能回放投票过程，而且并发下不需要行锁冲突。
- **关键关系**：N:1 → `purchase_proposals` / `users` / `poll_options`（选项模式）
- **核心约束**：`UNIQUE(voter_user_id, client_request_id)` 幂等键——弱网下用户重复点"驳回"不会投出两票，这是移动端最容易出的 bug；`CHECK` 保证 `choice` 与 `option_id` 不会同时有值

proposal_watches —— 围观

- **用途**：记录"我关注了这条审批但还没表态"。
- **为什么需要这张表**：界面上"围观"是一个独立按钮，个人页还有"我围观的"列表。围观者的心理是"我想看看结果，但现在不想站队"。这是真实需求，且**不能复用投票表**——否则计票会算错。
- **关键关系**：N:1 → `purchase_proposals` / `users`
- **核心约束**：`PRIMARY KEY(proposal_id, user_id)`

proposal_comments —— 回应

- **用途**：存广场和详情页的文字回应，支持一层楼中楼。
- **为什么需要这张表**：个人页有"收到回应"统计，广场文案是"给点你的想法"。投票只能表达"买/不买"，但用户常常想说"这个价位不如买 XX"——这是产品最有价值的 UGC 内容。`parent_comment_id` 只做一层，不做无限嵌套，是刻意的复杂度控制。
- **关键关系**：N:1 → `purchase_proposals` / `users`；自关联 → `proposal_comments`
- **核心约束**：`comment_status` 软删除，保留审计与统计口径

proposal_updates —— 作者补充

- **用途**：存发布后作者追加的说明，带独立时间戳。
- **为什么需要这张表**：详情页有一个独立的"作者补充"区块，时间显示为"2026-11-03 15:22"，**与主帖发布时间不同**。主表的 `purchase_reason` 是发布时写死的，改它等于篡改历史；补充说明是"事后追加"的语义，必须单独存。
- **关键关系**：N:1 → `purchase_proposals`
- **核心约束**：无特殊约束，一条审批可多次补充

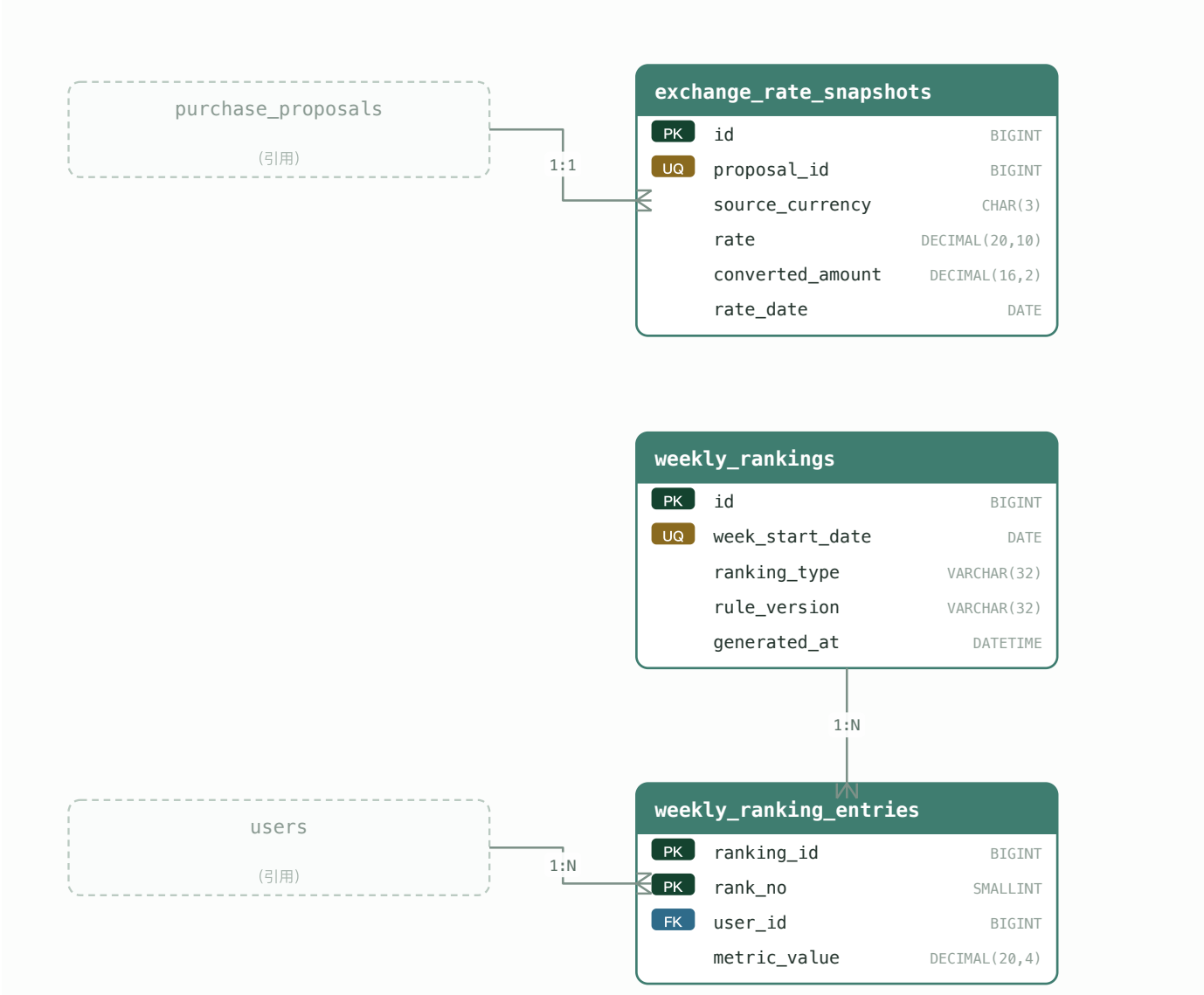
flowers —— 送花

- **用途**：记录谁给哪条内容送了花。
- **为什么需要这张表**：首页三宫格的第三个统计是"**收到花总数**"，这是产品的正向激励设计。原设计完全没有这个实体，但界面上明确存在，属于必须补的缺口。
- **关键关系**：N:1 → `purchase_proposals` / `users`（送花人、收花人各一个外键）
- **核心约束**：`UNIQUE(proposal_id, giver_user_id)` —— 同一人对同一条内容只能送一次，天然防刷

3.6 汇率与榜单域（3 张）

这个域解决什么：让"批准金额最大"这类榜单在跨币种（HKD / CNY / USD）时仍然可比，并且历史榜单能复现。

为什么需要汇率快照而不是实时汇率：假设上周榜单第一名是"¥5000 的相机"，本周汇率变了，如果按实时汇率重算，这条记录折算出的港币金额就变了，上周的榜单排名会和当时看到的不一样。用户会投诉"我明明是第一名"。**结算瞬间冻结汇率**，才能保证历史结论永不改变。



exchange_rate_snapshots —— 汇率快照

- **用途**：审批结算时，冻结当时的汇率和折算后的港币金额。
- **为什么需要这张表**：榜单的核心指标是"批准金额最大"，而审批可能是 HKD、CNY 或 USD。要放在一起排名必须统一折算。快照让这个折算结果**永久可复现**，不受后续汇率波动影响。
- **关键关系**：1:1 → `purchase_proposals`（`proposal_id` 唯一）
- **核心约束**：`UNIQUE(proposal_id)`；`rate > 0`；`converted_amount >= 0`

weekly_rankings —— 周榜快照头

- **用途**：记录某一周、某一类榜单的生成信息（周起始日、榜单类型、规则版本）。

- **为什么需要这张表：仅当需要"回看历史周榜"时才建**。如果只展示"当前周榜"，实时聚合就够了，这张表可以不要。但只要产品说"我想看上周榜"，就必须有快照——因为规则会变、用户会注销、内容会删除，事后重算的结果和当时看到的一定不一样。
- **关键关系**：1:N → `weekly_ranking_entries`
- **核心约束**：`UNIQUE(week_start_date, ranking_type)`；`week_start_date` 必须按 UTC+8 切周

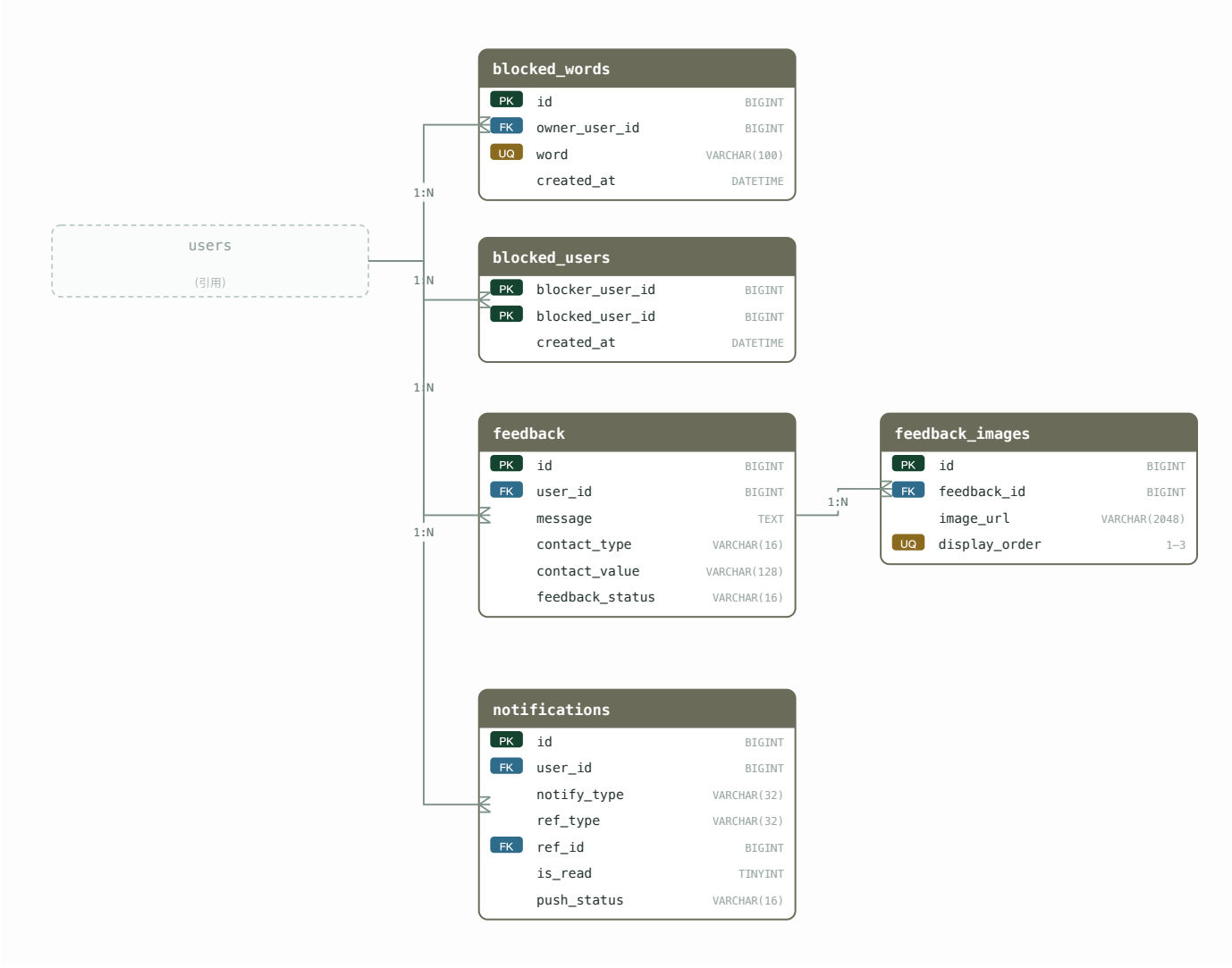
weekly_ranking_entries —— 周榜明细

- **用途**：存某期榜单的每一行：第几名、谁、指标值多少。
- **为什么需要这张表**：榜单头表只说明"这是哪一期"，具体排名必须有明细行。`metric_value` 用 `DECIMAL(20,4)` 是为了兼容两种量纲——金额（元）和次数（次）。
- **关键关系**：N:1 → `weekly_rankings` / `users`
- **核心约束**：`PRIMARY KEY(ranking_id, rank_no)` —— 同榜名次天然不重复

3.7 治理与运营域（5 张）

这个域解决什么：让用户能管理自己不想看的内容，让团队能收到反馈，让系统能主动提醒。

为什么屏蔽词要按用户存：界面上叫"屏蔽词库"，是**个人偏好**，不是平台审核。张三屏蔽"奶茶"，不影响李四看到。所以主键必须带 `owner_user_id`，不能用全局词表。



blocked_words —— 屏蔽词库

- **用途**：存每个用户自定义的屏蔽词，命中的内容在其信息流中隐藏。
- **为什么需要这张表**：对应"我的 – 屏蔽词库"入口。这是个人化的内容过滤，与平台级敏感词审核是两回事，不能混用一张表。
- **关键关系**：N:1 → `users`
- **核心约束**：`UNIQUE(owner_user_id, word)`；`word` **统一存小写**，否则 MySQL（大小写不敏感）和客户端 SQLite（敏感）的唯一性行为会不一致

blocked_users —— 黑名单

- **用途**：记录 A 拉黑了 B。

- **为什么需要这张表**：对应"我的 – 黑名单"。被拉黑者的内容不再出现在拉黑者的信息流里。这是 UGC 产品的标配，成本极低但缺失时体验很差。
- **关键关系**：N:M → `users`（自关联，双向各一个外键）
- **核心约束**：`PRIMARY KEY(blocker_user_id, blocked_user_id)`；`CHECK` 禁止拉黑自己

feedback —— 意见反馈

- **用途**：存用户提交的反馈内容、联系方式、处理状态。
- **为什么需要这张表**：对应"意见反馈"页面。它是产品迭代的输入来源，也是课程项目里"运营闭环"的体现。`user_id` 允许为空是为了用户注销后反馈内容仍可保留。
- **关键关系**：N:1 → `users`；1:N → `feedback_images`
- **核心约束**：`contact_type` 与 `contact_value` 分开存，便于按渠道统计回复率

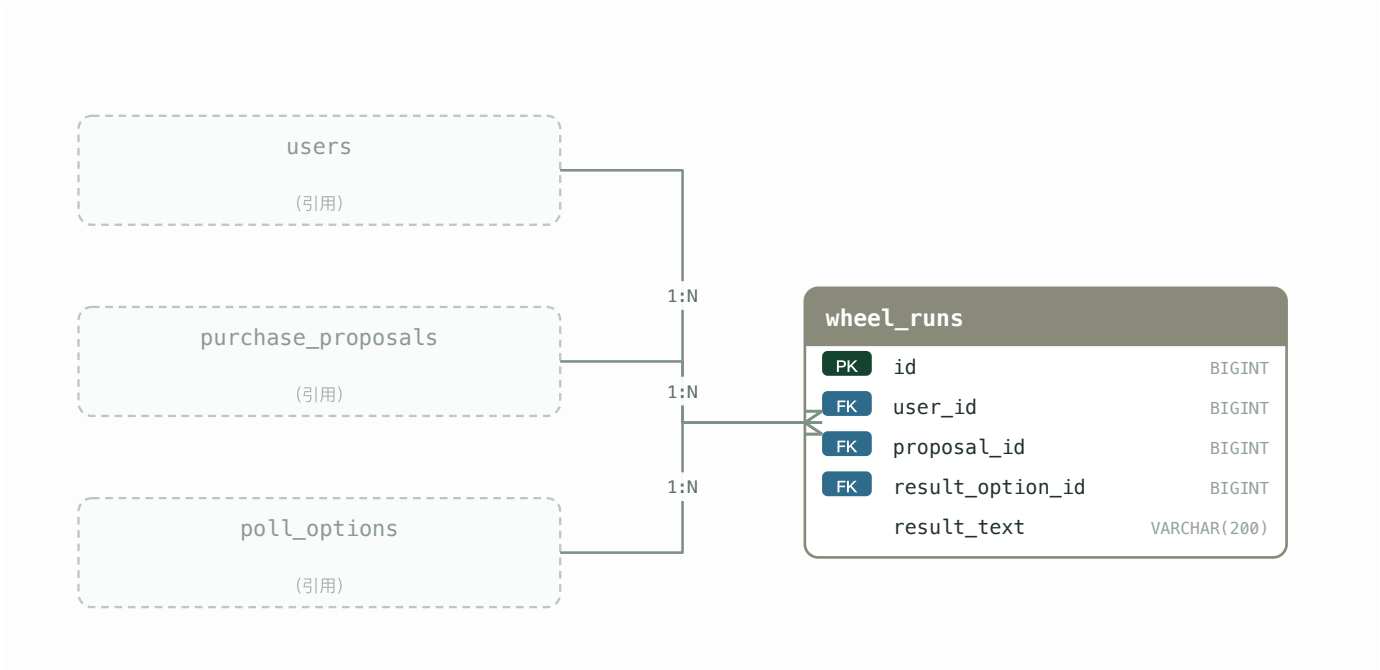
feedback_images —— 反馈截图

- **用途**：存反馈附带的截图，最多 3 张。
- **为什么需要这张表**：界面上明确有"上传图片（最多 3 张）"。用户报 bug 时一张截图胜过一百字描述，原表没有地方存，属于必须补的缺口。
- **关键关系**：N:1 → `feedback`
- **核心约束**：`UNIQUE(feedback_id, display_order)`，限 1–3

notifications —— 站内通知

- **用途**：存"你的审批有结果了""有人回应你""有人送你花"这类提醒。
- **为什么需要这张表**：FCM 是**尽力送达**，可能失败、可能被系统拦截。如果只在推送时发一次，丢了就永远丢了。落库后可以补发、可以去重、可以算未读数（首页红点）。
- **关键关系**：N:1 → `users`；多态关联 → `purchase_proposals` / `app_groups` / `proposal_comments`
- **核心约束**：`ref_type` + `ref_id` 多态设计，避免为每种通知建一张表

3.8 可选域（P2，1 张）



wheel_runs —— 转盘历史

- **用途**：记录每一次转盘抽取的结果。
- **为什么默认不建**：转盘的扇区直接来自 `poll_options`，抽取逻辑纯前端（`Canvas` 或 `Lottie` 动画）。只有产品明确要求"保存转盘历史"或"群内共享固定转盘"时，才需要这张表。
- **关键关系**：N:1 → `users` / `purchase_proposals` / `poll_options`（后两者可为空，表示自由转盘）
- **核心约束**：无

> **判断依据**：能不加表就不加表。加一张表容易，论证"不需要加"才是设计能力。

四、关系总览

4.1 一对多（1:N）

父表	子表	外键	业务含义
users	auth_sessions	user_id	一个用户多设备登录
users	user_devices	user_id	一个用户多台推送设备
users	app_groups	owner_user_id	一个人可以建多个群
users	group_members	user_id	一个人可以在多个群
users	purchase_proposals	creator_user_id	一个人发起多条审批
users	decision_votes	voter_user_id	一个人投多票
users	proposal_comments	author_user_id	一个人多条回应

父表	子表	外键	业务含义
users	feedback	user_id	一个人多条反馈
users	notifications	user_id	一个人多条通知
users	blocked_words	owner_user_id	一个人多个屏蔽词
app_groups	group_members	group_id	一个群多个成员
app_groups	purchase_proposals	group_id	一个群多条审批
channels	user_channels	channel_id	一个频道多人订阅
channels	purchase_proposal_channels	channel_id	一个频道多条审批
tags	proposal_tags	tag_id	一个标签多条审批
purchase_proposals	poll_options	proposal_id	一条审批多个选项
purchase_proposals	purchase_proposal_images	proposal_id	一条审批多张图
purchase_proposals	purchase_proposal_channels	proposal_id	一条审批多个频道
purchase_proposals	decision_votes	proposal_id	一条审批多张票
purchase_proposals	proposal_watches	proposal_id	一条审批多人围观
purchase_proposals	proposal_comments	proposal_id	一条审批多条回应
purchase_proposals	proposal_updates	proposal_id	一条审批多次补充
purchase_proposals	flowers	proposal_id	一条审批多朵花
poll_options	decision_votes	option_id	一个选项多票
decision_votes	reputation_ledger	source_vote_id	一票最多一条流水
weekly_rankings	weekly_ranking_entries	ranking_id	一期榜单多行
feedback	feedback_images	feedback_id	一条反馈多张图
proposal_comments	proposal_comments	parent_comment_id	一层楼中楼

4.2 一对一（1:1）

主表	从表	说明
users	reputation_accounts	`user_id` 既是主键又是外键，天然一人一行
users	user_streaks	同上
purchase_proposals	exchange_rate_snapshots	`proposal_id` 唯一，一条审批一个快照

4.3 多对多（N:M）

关系	中间表	说明
用户 ↔ 频道	user_channels	订阅关系，带排序
审批 ↔ 频道	purchase_proposal_channels	一稿多频道，共享一个票池
审批 ↔ 标签	proposal_tags	一条审批 1–3 个标签
用户 ↔ 用户	blocked_users	拉黑关系（自关联）

五、七个关键设计决策

决策 1：草稿和正式审批用同一张表

做法：`purchase_proposals.proposal_status = 'DRAFT'`。

理由：草稿和正式审批的字段重合度超过 90%（标题、金额、理由、图片、选项都一样），唯一区别是"有没有发布"。分成两张表会带来：两套 Entity、两套 Mapper、发布时要写迁移代码、字段改动要改两处。

代价：所有查询必须带 `WHERE proposal_status <> 'DRAFT'`。用一个 Mapper 层的公共条件就能解决。

决策 2：投票表设计成追加写入（append-only）

做法：改票不更新原行，而是插入新行；计票时取每人 `MAX(id)` 那一行。

理由： – **可审计**：能回答"这个人改过几次票、从什么改成什么" – **无并发冲突**：两个人同时投不同票，插入互不阻塞 – **历史可回放**：结算后还能复盘投票过程

代价：查询要用子查询取最新票。用索引 `(proposal_id, voter_user_id, id)` 可以高效支持。

决策 3：二元裁决和多选一共用 `decision_votes`

做法：加 `vote_type`（BINARY / OPTION）区分，`choice` 和 `option_id` 互斥。

理由：两者的"投票"行为完全一致——都是"某人、某时刻、对某条审批、基于某个票权、投了一次"。如果分成两张表，结算逻辑要写两遍，信誉发分要写两遍，个人页"我投过的"要 UNION 两次。

代价：需要一个 `CHECK` 约束保证字段互斥。这个代价很小。

决策 4：信誉分同时有汇总表和流水表

做法：`reputation_accounts`（当前分）+ `reputation_ledger`（明细）。

理由：这是"快照 + 流水"的标准组合。只存汇总，用户问"为什么扣分"答不出来；只存流水，每次投票算票权都要扫全表。两张表各解决一个问题。

关键：`reputation_ledger.source_vote_id` 的**唯一约束是幂等的最后一道防线**——即使定时任务重复跑，第二次插入会因唯一键冲突失败，不会重复发分。

决策 5：结算时冻结汇率

做法：`exchange_rate_snapshots` 在结算瞬间写入，之后永不更新。

理由：见 3.6 节。核心是"历史结论不可变"——这是所有涉及排名的系统都必须遵守的原则。

决策 6：计数冗余（`voter_count` / `watch_count` / `comment_count`）

做法：在主表冗余存三个计数字段，事务内更新。

理由：广场列表一页 20 条卡片，每条都要显示"已有 8 人表态""12 人围观""3 条回应"。如果实时算，就是 20 次 `COUNT(DISTINCT)` 子查询，列表会明显卡顿。冗余计数字段把 N 次聚合变成 0 次。

代价：需要保证一致性——所有增删操作必须在**同一个事务**里同时改明细和计数。

决策 7：转盘不建表

做法：扇区直接读 `poll_options`，抽取逻辑在前端。

理由：转盘不是独立功能，它是"选项投票"的第二种玩法。同一个 A/B/C 列表，既可以投票决定，也可以转盘决定。复用 `poll_options` 的好处是**数据闭环**——转盘的结果可以直接提交成一票，不会产生孤儿数据。

什么时候才需要建表：产品要求"保存转盘历史"或"群内共享固定转盘"时，加 `wheel_runs`。

六、建表顺序（依赖拓扑）

MySQL 有外键约束，建表必须按依赖顺序。以下顺序可直接用于 `V1__init.sql`：

批次	表	依赖
1	users	无
2	channels、tags	无
3	app_groups	users
4	group_members	app_groups、users
5	purchase_proposals	users、app_groups
6	poll_options、purchase_proposal_images、purchase_proposal_channels、proposal_tags	purchase_proposals、channels、tags
7	decision_votes	purchase_proposals、users、poll_options
8	proposal_watches、proposal_comments、proposal_updates、flowers	purchase_proposals、users
9	reputation_accounts、reputation_ledger	users、decision_votes
10	exchange_rate_snapshots	purchase_proposals
11	weekly_rankings → weekly_ranking_entries	users
12	auth_sessions、user_devices、user_streaks	users
13	blocked_words、blocked_users、feedback → feedback_images、notifications	users
14	wheel_runs（P2）	users、purchase_proposals、poll_options

> `user_channels` 可放在批次 3，依赖 users 和 channels。

七、与原设计的差异

处理	说明
保留 9 张	channels、group_members、purchase_proposal_channels、reputation_accounts、exchange_rate_snapshots、blocked_words、blocked_users 等，结构与约束完全不动
修改 6 张	users、app_groups、purchase_proposals、decision_votes、feedback、purchase_proposal_images
新增 15 张	业务侧 9 张 + 安卓侧 3 张 + 运营侧 1 张 + P2 2 张
删除字段 1 个	`app_groups.invite_code_hash` —— 哈希不可反查，与"输入群号加入"的功能直接冲突

整体评价：原设计的表结构规范性很好——`DECIMAL` 金额、`DATETIME(6)` 存 UTC、`CHECK` 约束、复合主键、append-only 投票、汇率快照、`rule_version` 都体现了工程素养。问题不在"多"，在"少"：它按"一个人问、几个人答买不买"这一条主线建模，而产品实际有三条主线（二元裁决 / 多选一 / 广场围观互动）。

文档版本：V1 | 技术栈：Spring Boot（Java）+ Android + MySQL 8.4 | 2026-11-03